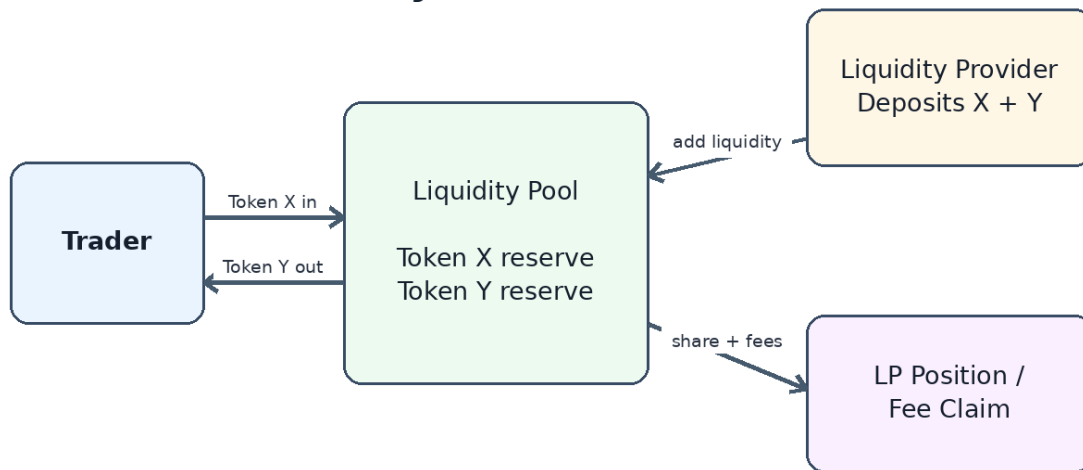


DAY 14

DeFi Fundamentals: DEXs, AMMs & Liquidity Pools

Complete 60-Minute Beginner-to-Freelancer Lecture

AMM Pool Anatomy



Goal: Understand how an onchain token swap works from wallet approval to pool math, execution, logs, and risk controls.

Course context: Days 1-13 built blockchain, Solidity, security, frontend and backend skills. Day 14 applies them to the most common DeFi building block: token exchange through automated liquidity pools.

1. Learning Objectives and 60-Minute Teaching Plan

By the end of Day 14, a beginner should be able to explain DeFi and DEX architecture, calculate a simple constant-product swap, distinguish spot price, price impact and slippage, describe liquidity-provider positions, trace a router-to-pool transaction, identify key integration risks, and translate a client DEX request into testable deliverables.

Time	Topic	Outcome
0-5 min	DeFi + DEX mental model	Know where swaps fit in the DeFi stack.
5-12 min	AMM and liquidity pools	Understand pool-based trading vs order books.
12-20 min	$x * y = k$ math	Calculate reserve changes and output amount.
20-27 min	Fees, price impact, slippage	Understand why quotes and final output differ.
27-34 min	Router / factory / pool architecture	Trace contracts used by a swap.
34-40 min	Approvals + swap parameters	Understand allowance, path, min output and deadline.
40-46 min	Liquidity providers	Add/remove liquidity and understand LP economics.
46-51 min	v2 -> v3 -> v4 mental model	Know why modern AMMs differ from the basic model.
51-56 min	Security + MEV + testing	Protect users and integrations.
56-60 min	Explorer lab + freelancer review	Analyze a real-style swap and client scope.

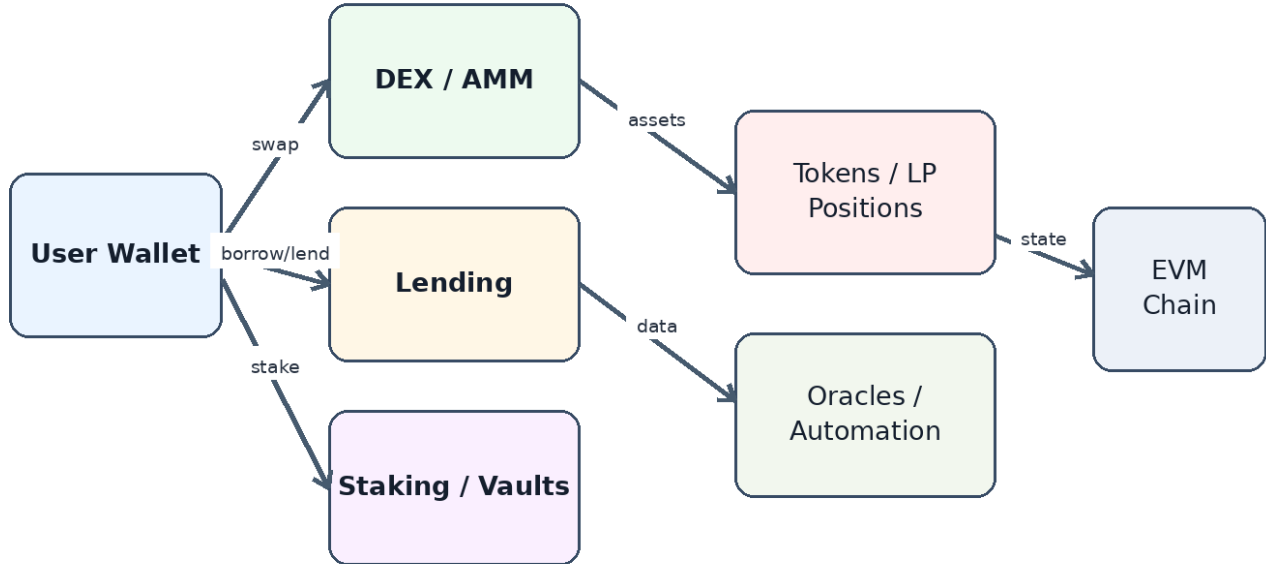
What you should be able to say in 60 seconds

Core explanation: A DEX is an onchain trading system. In an AMM, traders exchange tokens against reserves held by smart-contract pools rather than waiting for matching orders. Pool pricing follows an invariant or other algorithm. The user normally approves token spending, calls a router with a path and protection parameters, the router executes against one or more pools, and the transaction succeeds only if the final output satisfies the user's limits.

2. What Is DeFi?

Decentralized finance (DeFi) refers to financial applications built from smart contracts and blockchain assets. Common categories include decentralized exchanges, lending, borrowing, staking, derivatives, stablecoins, asset management, payments and automated vaults. The important developer idea is composability: one protocol can call or hold assets from another protocol in the same transaction.

DeFi: Composable Financial Smart Contracts



2.1 DeFi vs traditional finance - developer view

Traditional service	DeFi analogue	Developer integration surface
Broker/exchange	DEX / AMM	Router, pool, quote API/SDK, events
Bank loan	Lending protocol	Markets, collateral, interest indexes, oracles
Investment fund	Vault / strategy	Deposit/withdraw shares, accounting
Payment processor	Onchain transfer/payment contract	Tokens, confirmations, events
Market data feed	Oracle network	Price feed interface + freshness checks

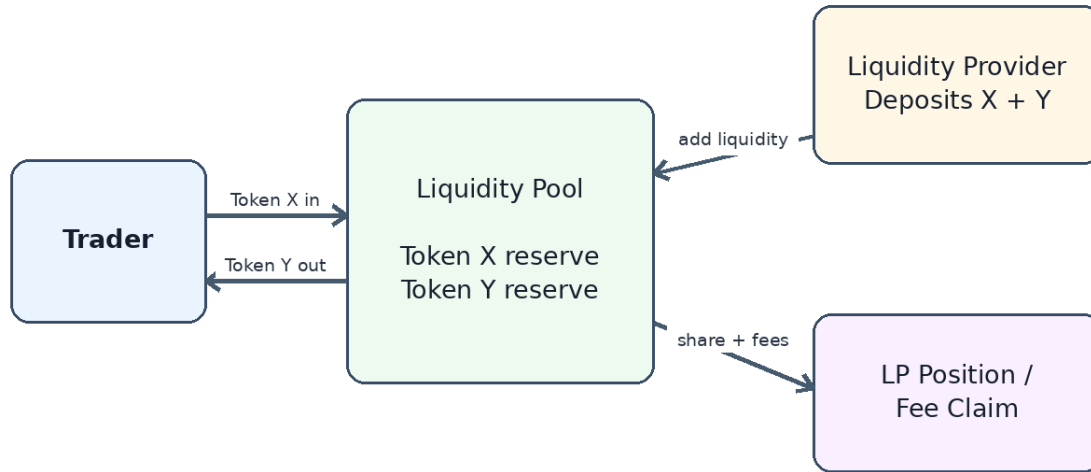
Important: DeFi contracts do not remove risk. They replace some organizational trust with code, economic mechanisms, governance, oracles and infrastructure. A freelancer must identify which assumptions remain.

3. Centralized Exchange Order Book vs AMM

Traditional and many centralized crypto exchanges use order books: buyers and sellers publish orders at prices and a matching engine executes compatible orders. An AMM instead allows a trader to interact directly with pooled reserves using deterministic smart-contract rules.

Question	Order book	AMM pool
Who provides liquidity?	Market makers and limit-order traders	Liquidity providers deposit assets
How is trade matched?	Against opposite orders	Against pool reserves
Price source	Order-book depth	Pool state + AMM formula
Execution environment	Usually centralized matching engine	Smart contracts onchain
Key developer risk	API/matching semantics	Slippage, pool math, token/contract behavior

AMM Pool Anatomy



3.1 Why AMMs became important

- Smart contracts can execute swaps deterministically without a centralized matching engine.
- Liquidity is shared by many users and becomes available through a common pool.
- Other smart contracts can compose swaps into larger onchain workflows.
- Price changes automatically as reserve ratios change.

4. Liquidity Pool Fundamentals

A two-token pool holds reserves of token X and token Y. The pool state is the source of truth for the AMM. When a trader sends X into the pool and receives Y, the reserve ratio changes. The AMM rule determines how much Y can leave without violating the pool invariant.

Example pool state

Token X reserve: 100 ETH

Token Y reserve: 200,000 USDC

Simple reserve ratio price:

$200,000 \text{ USDC} / 100 \text{ ETH} = 2,000 \text{ USDC per ETH}$

4.1 Spot/reserve ratio is not your guaranteed execution price

The 2,000 USDC/ETH ratio is useful for intuition, but a trade changes the reserves while it executes. Larger trades relative to the pool cause a larger average execution-price change. Fees also reduce the amount of input used by the invariant calculation.

4.2 Who supplies the reserves?

Liquidity providers (LPs) deposit assets into the pool. In exchange they receive a position that represents a claim on pool liquidity and fee earnings. The exact representation depends on the AMM version: a v2-style pool can use fungible LP tokens, while concentrated-liquidity systems track individual positions and ranges.

Freelancer vocabulary: Pool depth or liquidity describes how much capital is available around the current trading price. Deep liquidity generally means a given trade has less price impact than the same trade against a shallow pool.

5. Constant-Product AMM: $x * y = k$

The classic Uniswap v2 teaching model uses a constant-product invariant. Let x be the reserve of token X and y the reserve of token Y. Ignoring fee details for one moment, a trade changes x and y while keeping their product approximately constant.

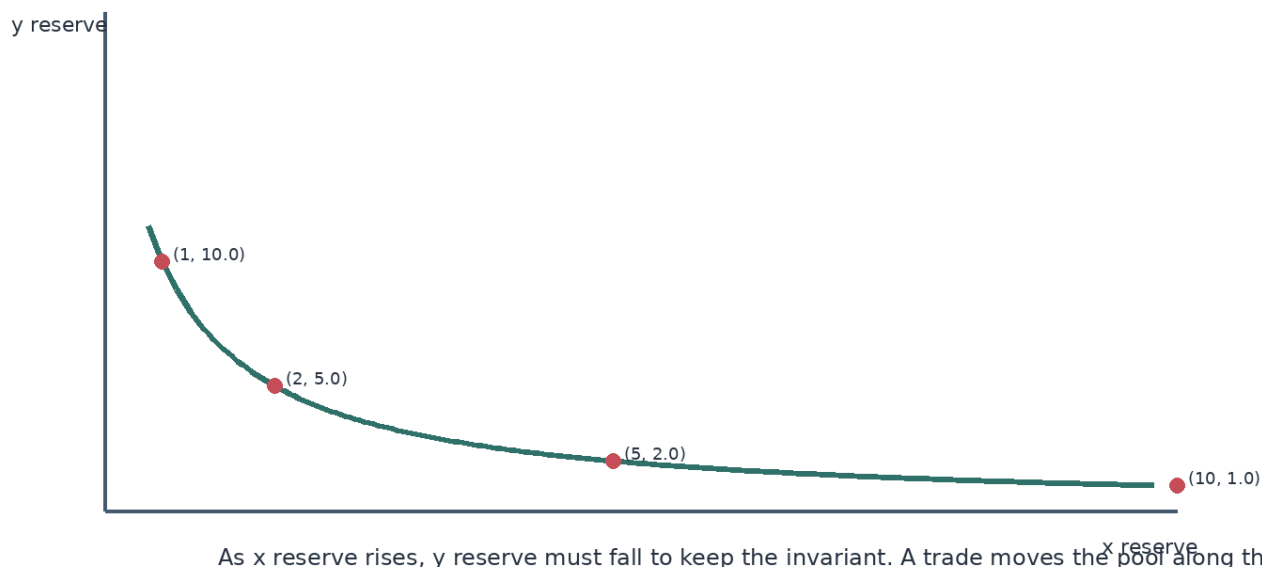
$$x * y = k$$

x = reserve of token X

y = reserve of token Y

k = invariant

Constant-Product Curve: $x * y = k$



As x reserve rises, y reserve must fall to keep the invariant. A trade moves the pool along the c

5.1 Intuition

- A trader adds one reserve and removes the other.
- The pool cannot sell all of one asset under the basic curve.
- As one reserve becomes scarce, its marginal price rises sharply.
- The curve itself creates price impact: execution changes the state that determines price.

Teaching boundary: Use $x*y=k$ to learn AMM mechanics. Modern AMMs can use concentrated liquidity, hooks, dynamic fees, alternative curves or additional routing logic, so production integrations must use the exact protocol's math/SDK/contracts rather than a hand-written simplified quote.

6. Worked Swap Calculation - Without Fee

Start with the same pool: 100 ETH and 200,000 USDC. The invariant is 20,000,000. Suppose a trader sends 10 ETH and, only for this first example, ignore swap fees.

Before:

```
x = 100 ETH
y = 200,000 USDC
k = 100 * 200,000 = 20,000,000
```

After 10 ETH enters:

```
x_new = 110 ETH

y_new = k / x_new
       = 20,000,000 / 110
       = 181,818.18 USDC
```

```
USDC out = 200,000 - 181,818.18
          = 18,181.82 USDC
```

Average execution price:

```
18,181.82 / 10 = 1,818.18 USDC per ETH
```

6.1 What happened to the price?

The starting reserve ratio suggested 2,000 USDC/ETH, but the trader received an average of about 1,818 USDC/ETH because a 10 ETH trade is large relative to a 100 ETH reserve. That difference is primarily price impact from moving the AMM curve.

Before	After
100 ETH reserve	110 ETH reserve
200,000 USDC reserve	181,818.18 USDC reserve
Reserve ratio ~2,000 USDC/ETH	New ratio ~1,652.89 USDC/ETH

7. Swap Fees - Worked v2-Style Example

A swap fee reduces the effective input used in the invariant calculation and compensates liquidity. For a simple v2-style arithmetic example using a 0.30% gross swap fee, 10 ETH becomes 9.97 ETH of effective input for the pricing formula.

```
amountIn = 10 ETH
fee = 0.30%
effectiveIn = 10 * 0.997 = 9.97 ETH

amountOut = reserveOut * effectiveIn
            -----
            reserveIn + effectiveIn

amountOut = 200,000 * 9.97 / (100 + 9.97)
           = 18,132.22 USDC (approximately)

Average output per 10 ETH input:
~1,813.22 USDC/ETH
```

The exact fee mechanism is protocol/version specific. For example, current Uniswap documentation distinguishes v2, v3 and v4 fee accounting, and v4 supports configurable and dynamic fee logic. Your integration should obtain fee configuration and quote logic from the protocol you are actually using.

7.1 Fee terms to separate

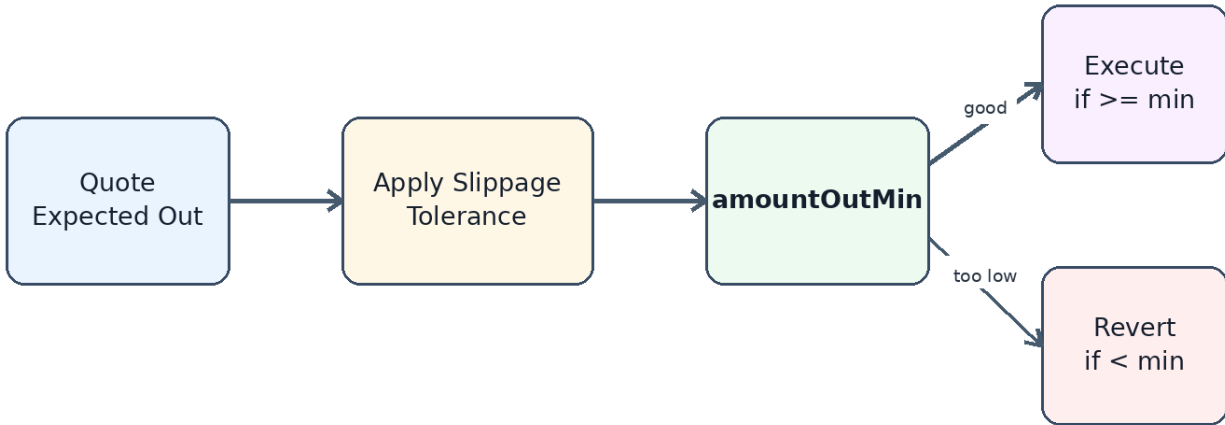
Term	Meaning
Swap fee	Fee charged by the pool/protocol during trade execution.
LP fee	Portion attributable to liquidity providers under protocol rules.
Protocol fee	Possible portion directed to protocol governance/treasury mechanism.
Network gas fee	Separate EVM transaction execution cost paid to the chain.

8. Price Impact vs Slippage

These terms are often confused in client conversations. Price impact is the price movement caused by your own trade against available liquidity. Slippage is the difference between an expected/quoted execution and the actual execution caused by state changes, routing, market movement, or transaction ordering before confirmation.

Concept	Cause	Developer control
Price impact	Trade size relative to liquidity and AMM curve	Choose route/pools; reduce trade size; aggregate liquidity
Slippage	State changes between quote and execution	Set minimum output / maximum input constraints
Gas fee	Network execution cost	Optimize route/code; choose network/timing
Token fee/tax	Token contract behavior	Detect/support explicitly or reject

Quote, Price Impact and Slippage Guard



8.1 amountOutMin

A common exact-input swap pattern includes a minimum acceptable output. If the swap would return less than that amount, the transaction reverts rather than silently accepting an arbitrarily bad execution.

```

quotedOut = 1,000 USDC
slippageTolerance = 0.5%

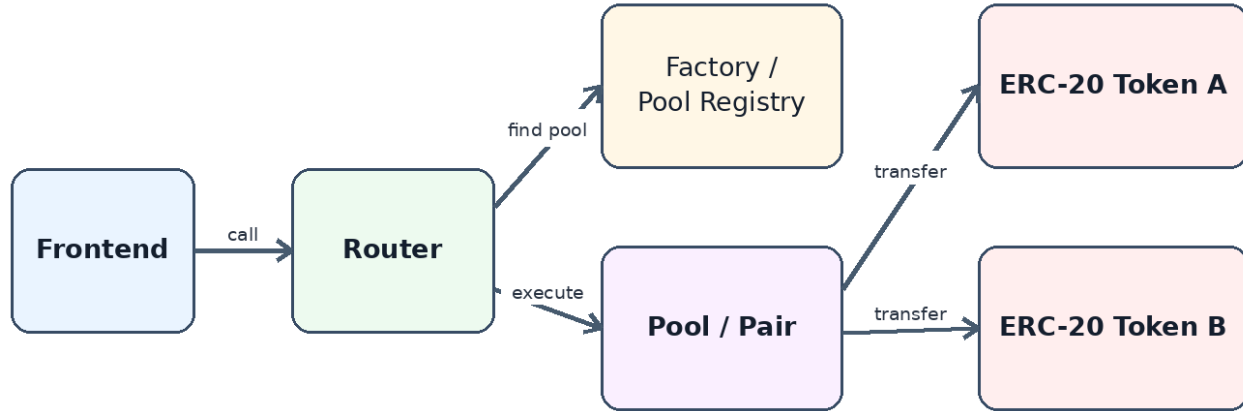
amountOutMin = 1,000 * (1 - 0.005)
              = 995 USDC
  
```

Security rule: Do not casually set amountOutMin to zero in production code. Removing the output floor weakens the user's protection against unexpected execution and adversarial transaction ordering.

9. DEX Contract Architecture: Router, Factory and Pool

DEX integrations rarely call one monolithic contract. A useful mental model is: the factory or registry identifies pools; the pool owns liquidity and executes AMM accounting; routers/periphery contracts provide safer and more convenient user-facing paths, multi-hop routing and token handling.

DEX Contract Mental Model



Component	Role	What a freelancer checks
Factory / registry	Creates or locates pools	Correct network/address; token ordering; pool existence
Pool / pair	Holds liquidity and executes AMM state changes	Reserves/liquidity, fee, callbacks, events
Router / periphery	Coordinates one or more pools	Function signature, path, approvals, deadlines, value handling
Quote/SDK layer	Calculates candidate route/output offchain	Version compatibility, decimals, fee assumptions
Permit/approval layer	Authorizes token movement	Spender address, amount, expiration, signature domain

10. Token Approval Before a Swap

For an ERC-20 exact-input swap, the user normally must authorize the contract that will pull the input token. In the simple allowance model this means `approve(spender, amount)`. Modern systems may also use permit-style signatures or approval-management contracts, but the principle is the same: the swap path needs authority to transfer the user's input token.

```
// ERC-20 allowance model
IERC20(tokenIn).approve(router, amountIn);

// Router later executes a transferFrom-like pull
// from user -> router/pool according to protocol design.
```

10.1 Approval questions to answer before coding

- Which contract is the actual spender: router, Permit2, position manager, vault, or another helper?
- Exact allowance or unlimited allowance? What is the product's risk policy?
- Does the token require setting allowance to zero before changing it?
- Is the token fee-on-transfer, rebasing, callback-enabled, or otherwise non-standard?
- Does the user need native ETH or WETH for the selected route?

Client-debugging clue: "Transfer amount exceeds allowance" usually means the token owner approved the wrong spender, approved too little, switched networks, or the frontend is reading allowance with the wrong owner/token/router combination.

11. Swap Parameters You Must Understand

The exact function names depend on protocol/version, but most swap APIs encode the same business intent: input token, output token, input or output amount, route/pool information, receiver, price-protection constraints, and sometimes a deadline.

Parameter concept	Why it exists	Typical bug
tokenIn/tokenOut	Defines assets	Wrong token address/network
amountIn	Exact input quantity	Wrong decimals / raw units
amountOutMin	Protects exact-input execution	Zero or stale value
amountOut	Exact desired output	Incorrect quote
amountInMax	Protects exact-output execution	Too high / unsafe user approval
path/route	Selects pools/hops	Bad token order or unavailable pool
recipient	Receives output	Wrong account or contract
deadline	Limits transaction lifetime where supported	Expired timestamp / wrong units

11.1 v2-style router interface for learning

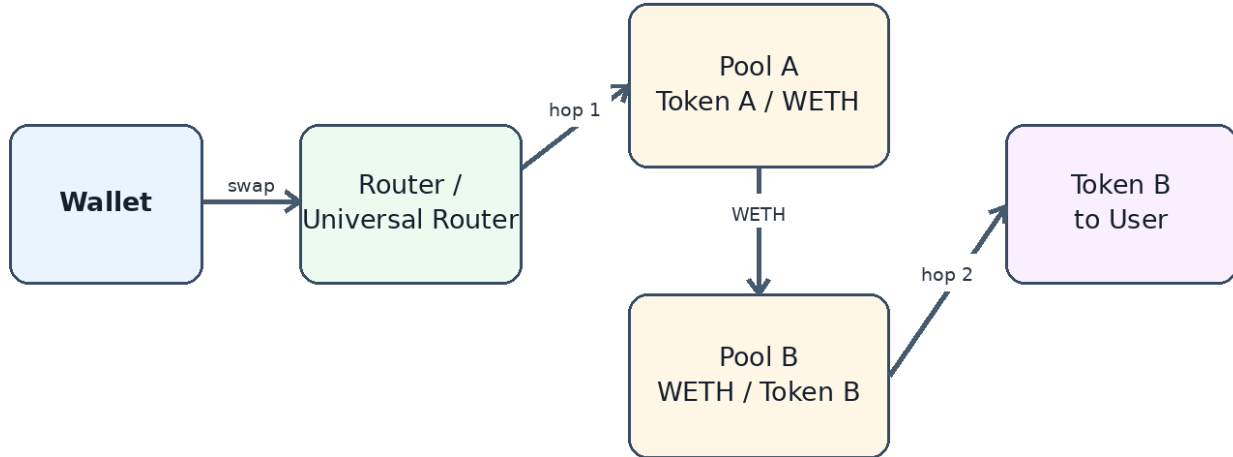
```
interface IV2RouterLike {
    function swapExactTokensForTokens(
        uint256 amountIn,
        uint256 amountOutMin,
        address[] calldata path,
        address to,
        uint256 deadline
    ) external returns (uint256[] memory amounts);
}
```

Important: This interface is a teaching example for v2-style routers. Do not assume it matches v3, v4, aggregators or every DEX clone. Always use the ABI and integration guide for the exact deployment.

12. Single-Hop vs Multi-Hop Routing

A direct pool may not be the best or even exist. Routers can connect pools through intermediate assets. For example, TOKEN_A -> WETH -> TOKEN_B uses two swaps inside one transaction. More hops can improve available liquidity but add gas, fees and complexity.

Typical DEX Swap Path



12.1 Route evaluation checklist

- Output after all pool fees, not just the first hop.
- Liquidity depth and price impact at each hop.
- Gas cost of the route.
- Token transfer quirks and wrapping/unwrapping of native assets.
- Minimum output computed for the entire route.
- Whether quote and execution use the same pool state/version assumptions.

Example route concept:

USDC -> WETH -> TOKEN

Hop 1 quote: USDC -> WETH

Hop 2 quote: WETH -> TOKEN

Final quote: TOKEN amount after both hops and fees

13. Analyze a Swap Transaction Like a Freelancer

When a client sends a transaction hash and says “the swap is wrong,” do not start by guessing at the frontend. Build an evidence trail from chain data.

1. Confirm chain ID/network and transaction status.
2. Identify sender and destination contract.
3. Decode the called function using the correct ABI.
4. Record exact input, minimum output, path/route, recipient and deadline-like fields.
5. Inspect ERC-20 Approval/Transfer events and pool swap events.
6. Measure actual output received by the recipient.
7. Compare quote block/time to execution block/time.
8. Check pool liquidity, fee, token decimals and token behavior.
9. Check whether the transaction was reordered or exposed to adverse MEV.
10. Write a concise cause statement backed by decoded data.

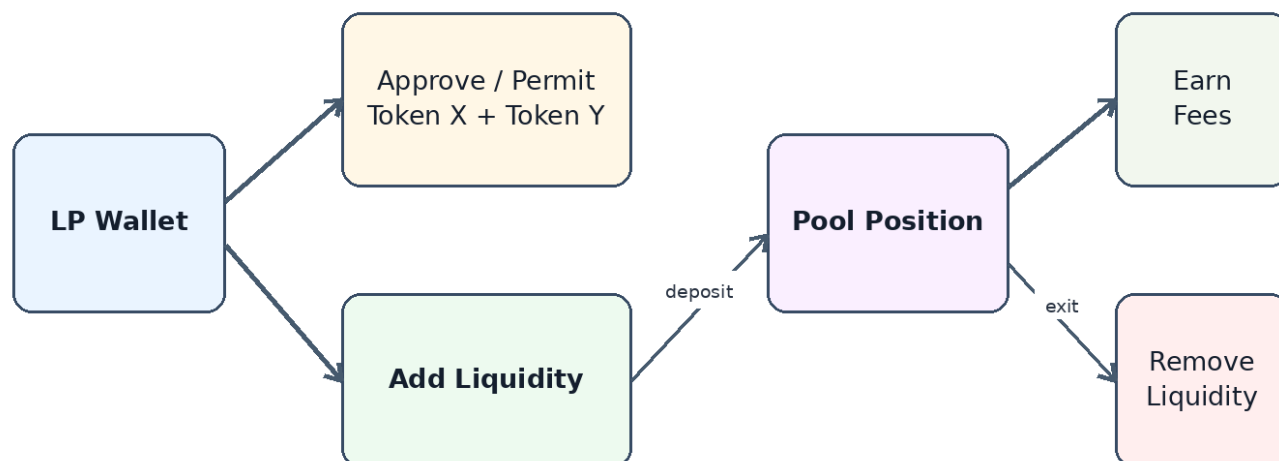
13.1 Transaction analysis worksheet

Field	Your finding
Network / chain ID	
Transaction hash	
Status / block	
Sender	
Router / entry contract	
Input token + raw amount	
Output token + actual amount	
Minimum output	
Route / pools	
Gas used / fee	
Key events	
Conclusion	

14. Add Liquidity: What the LP Is Doing

A liquidity provider supplies assets to make swaps possible. In a simple full-range two-token model, the LP deposits both assets at the pool's current ratio and receives a proportional position. Concentrated-liquidity AMMs instead allow an LP to choose a price range and manage a more individualized position.

Liquidity Provider Lifecycle



14.1 v2-style liquidity intuition

Pool before:

100 ETH + 200,000 USDC

LP wants 10% of current reserves worth of new liquidity:

10 ETH + 20,000 USDC

After deposit (simplified):

110 ETH + 220,000 USDC

LP receives a position representing the deposit's pool share.

14.2 Remove liquidity

Removing liquidity burns/redeems the LP position and returns the LP's share of the pool reserves, including the economic effect of accumulated fees and price changes. The LP may receive a different token mix from the original deposit because traders have changed the reserves.

15. LP Economics and Impermanent Loss - Beginner Model

Liquidity provision is not the same as simply holding two tokens. As market prices move and arbitrage rebalances the AMM, the pool changes the quantity of each asset held by the LP position. Compared with holding the original quantities, the LP position can underperform; this difference is commonly described as impermanent loss. Fees may offset some or all of that difference, but there is no guarantee.

15.1 What changes LP results?

- Relative price movement between the paired assets.
- Trading volume and fee income.
- Chosen fee configuration and protocol fee rules.
- Liquidity concentration/range management in modern AMMs.
- Time spent in range for concentrated liquidity.
- Token-specific risks, incentives and rewards.
- Gas costs for position management.

Teaching boundary: This course explains the mechanism for development work; it is not investment advice. When building LP dashboards, show inputs, assumptions, fees and risk metrics rather than promising returns.

15.2 Developer metrics for an LP dashboard

Metric	Source
Position liquidity/share	Pool/position contract
Token amounts	Pool math + position state
Unclaimed fees	Protocol position accounting
Current price / tick	Pool state
In-range status	Current price vs selected range
Gas spent	Transaction receipts
Historical PnL estimate	Indexed state + explicit methodology

16. Uniswap v2, v3 and v4 - The Mental Model

The constant-product model remains the best first AMM lesson, but current production integrations may use newer architectures. You should know the conceptual progression without trying to memorize every implementation detail on Day 14.

Version	Beginner mental model	Integration consequence
v2	Full-range two-token constant-product pools; fungible LP tokens	Simple reserve math and router-style flows; useful for fundamentals/legacy integrations
v3	Concentrated liquidity within chosen price ranges; positions represented individually	Ticks/ranges, position NFTs, fee tiers and more complex quote/liquidity math
v4	Concentrated liquidity with singleton PoolManager, hooks, flash accounting and configurable fee behavior	New integration architecture; hooks can customize lifecycle behavior; use current v4 tooling/docs

16.1 Why this matters to a freelancer

- A client saying “Uniswap integration” is not enough; ask which protocol version and chain deployment.
- Do not paste a v2 router ABI into a v3/v4 task.
- LP positions, fee collection and routing semantics differ materially by version.
- Modern periphery and approval tooling can be separate from the pool contracts.

Current integration note: Uniswap's current developer overview identifies v4 as the recommended starting point for new Uniswap integrations, while v2 remains useful for fundamentals and legacy systems.

17. Modern Routing, Approvals and Periphery

Production DEX integrations may involve more than a pool and a router. Modern stacks can include universal routers, permit/approval systems, position managers, quote services, SDKs, subgraphs/indexers and hooks. Treat each component as a separate trust and compatibility boundary.

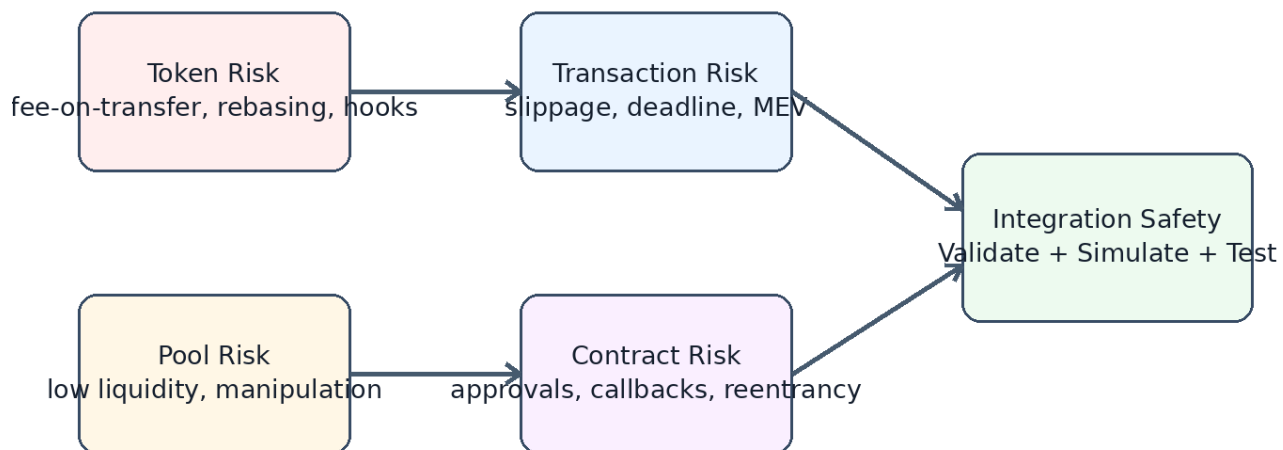
Layer	Question to ask
Quote/route engine	Where did the quote come from and at what block/state?
Approval manager	Who is authorized to transfer the user token, for how long and how much?
Router/periphery	Which actions are executed and in what order?
Pool manager/pools	Which fee and liquidity rules apply?
Hook/plugin	Can external custom logic run before/after a pool action?
Indexer/subgraph	Is displayed data eventually consistent or chain-authoritative?

17.1 Integration rule

Never infer contract behavior from a UI label: Resolve the exact chain, version, deployed addresses, ABI, token addresses, fee/pool configuration and official integration method before writing transaction code.

18. DEX Security and Integration Risks

DEX Integration Risk Layers



18.1 High-value checks

- Use trusted, chain-correct deployed addresses; never accept an unverified router address from arbitrary client input.
- Validate token addresses and decimals; use raw integer units correctly.
- Set realistic minimum-output or maximum-input protection.
- Avoid unrestricted approvals when the product does not require them; display approval scope clearly.
- Handle tokens with non-standard transfer behavior deliberately rather than assuming all ERC-20s behave identically.
- Protect callbacks/external calls against reentrancy and unexpected caller behavior.
- Do not trust a pool spot price as a robust oracle for unrelated lending/accounting decisions.
- Simulate and test the exact route on a fork/test environment before production rollout.

18.2 Pool price is not automatically an oracle

A pool price can move materially within a transaction, especially in shallow liquidity. Systems that need a price oracle should use an oracle design appropriate to their threat model rather than blindly reading a manipulable instantaneous pool ratio.

19. MEV, Sandwich Risk and Transaction Ordering

DEX swaps are sensitive to transaction ordering because the state of a liquidity pool determines execution. Public transaction propagation can expose pending swaps to searchers. A common adverse pattern is sandwich trading: an attacker trades before a victim to move the price, the victim executes at a worse price within their tolerance, and the attacker trades back after.

Simplified ordering:

1. Victim broadcasts large swap
2. Searcher buys first -> pool price moves
3. Victim swap executes at worse price
4. Searcher sells after victim -> captures value

19.1 Developer mitigations and design questions

- Keep slippage tolerance no wider than product requirements.
- Avoid zero minimum output.
- Use fresh quotes and expire stale transaction intents where the protocol supports it.
- Consider private/protected transaction submission where appropriate and available.
- Warn users when price impact is high.
- Split/route trades only when economically justified and tested.
- Do not promise that a frontend setting completely eliminates MEV.

Freelancer deliverable: For a DEX frontend, “slippage support” is not just a settings box. It must feed into the actual transaction constraint and be tested against quote changes and revert conditions.

20. Integration Testing Strategy

A correct DEX integration should be tested at several layers. A mocked unit test is not enough because the most important failures come from token behavior, pool state, contract version, approvals and route execution.

Test layer	Example
Pure/math unit	Raw units, slippage calculation, percentage basis points
Contract unit	Access control, wrapper validation, callbacks
Fork integration	Real deployed router/pool/token behavior at pinned block
Frontend integration	Wallet, network, approval, quote, pending/success/revert states
Backend/indexer	Decode swap events and reconcile actual token transfers
Adversarial	Stale quote, insufficient allowance, low liquidity, high impact, reorg/retry

20.1 Example slippage helper test

```
// JavaScript example
function minOut(quote, slippageBps) {
  return quote * BigInt(10_000 - slippageBps) / 10_000n;
}

console.assert(minOut(1000n, 50) === 995n); // 0.50%
```

20.2 Fork-testing questions

- Pinned block number for reproducibility?
- Correct network and router/pool deployment?
- Does impersonated test account have required token balances?
- Approval performed to the correct spender?
- Do emitted transfers match the reported output?
- Does the transaction revert when amountOutMin is intentionally too high?

21. Hands-On Lab: Build a Safe Swap Analyzer

For Day 14, the beginner lab is analysis-first rather than risking funds. Use a testnet or a pinned local fork. The goal is to connect all previous course skills: RPC, ABI decoding, ERC-20 events, AMM math, gas, and security.

Lab A - Explorer / RPC transaction analysis

1. Choose a known successful DEX swap on the intended test environment.
2. Record chain, hash, block, sender and destination contract.
3. Decode input parameters with the exact router ABI.
4. Identify input/output token addresses and decimals.
5. Read Approval and Transfer logs.
6. Identify the pool(s) and swap-related events.
7. Calculate actual input and output from transfers/events.
8. Compare actual output with transaction protection parameters.
9. Write a five-sentence client-style conclusion.

Lab B - Local constant-product simulator

```
def amount_out(amount_in, reserve_in, reserve_out, fee_bps=30):
    fee_denominator = 10_000
    amount_in_with_fee = amount_in * (fee_denominator - fee_bps)
    numerator = amount_in_with_fee * reserve_out
    denominator = reserve_in * fee_denominator + amount_in_with_fee
    return numerator // denominator

# Integer-unit simulation only; protocol-specific production code
# must use the exact AMM's official math/contracts/SDK.
```

Lab C - Safety assertions

- Reject zero token addresses.
- Reject tokenIn == tokenOut.
- Reject amountIn == 0.
- Reject quote that produces zero output.
- Apply bounded slippage configuration.
- Confirm chain ID before transaction construction.

22. Upwork / LinkedIn Client Scenarios

Scenario 1 - “Add token swap to our React dApp”

Deliverables: network/version confirmation, official address configuration, wallet/network UX, token approval flow, quote display, price-impact warning, slippage setting, transaction construction, pending/success/revert UI, explorer link, tests and README.

Scenario 2 - “Swap transaction keeps reverting”

Debug sequence: decode revert if possible -> verify allowance -> raw token units -> route/pool existence -> minimum output -> deadline -> token behavior -> native-value requirements -> router/version mismatch -> simulate on fork.

Scenario 3 - “Build a token launch with liquidity”

Clarify whether the client means pool creation, initial liquidity, price/range configuration, position ownership, lock/vesting requirements, fee settings, admin rights and post-launch operations. Do not assume “add liquidity” is the whole scope.

Scenario 4 - “Create arbitrage/MEV bot”

Before implementation, separate educational research from production capital deployment. Define chain, DEX versions, data latency, private transaction strategy, simulation, gas model, failure handling and legal/compliance constraints. Profitability is not guaranteed, and well-known opportunities are highly competitive.

22.1 Proposal language

Strong proposal structure: “I will first verify the target chain, DEX version, deployed contracts and token behavior. Then I will implement quote + approval + protected swap execution, test against forked chain state, add transaction/error UX, and document configuration and security assumptions.”

23. Interview Questions - Practice Out Loud

Question	Strong short answer
What is an AMM?	A smart-contract market that prices trades algorithmically against pooled liquidity rather than matching an order book.
What does $x*y=k$ mean?	A constant-product invariant linking the two reserves in a classic two-token AMM.
Price impact vs slippage?	Price impact comes from your trade moving the pool; slippage is quote-to-execution difference and is bounded with transaction constraints.
Why approve before swapping ERC-20?	The swap spender needs allowance or equivalent permission to pull the input tokens.
Why use amountOutMin?	It reverts an exact-input swap if execution would return less than the user accepts.
What is a router?	A periphery contract that coordinates token handling and one or more pool interactions.
What is an LP?	A liquidity provider supplying assets to pools in exchange for a position and protocol-defined fee economics.
Why not use pool spot price as an oracle?	It may be manipulable within a transaction or shallow market; oracle design needs stronger assumptions.
Why fork test a DEX integration?	It validates exact deployed contracts, tokens and live-like pool behavior without risking production funds.
Why ask protocol version?	v2/v3/v4 architectures, ABIs, positions, fee logic and routing differ materially.

23.1 Mini Quiz

1. A pool has 50 ETH and 100,000 USDC. What is the simple reserve-ratio price?
2. If a trader is large relative to pool reserves, does price impact generally rise or fall?
3. What does a zero amountOutMin remove?
4. What is the difference between an ERC-20 approval and the swap itself?
5. Name two reasons a quote may differ from final execution.
6. Why can a multi-hop route beat a direct route?
7. What happens to an LP token mix as traders rebalance the pool?
8. What does a transaction receipt prove that a frontend "submitted" message does not?

Quiz answers

1) 2,000 USDC/ETH. 2) Rise. 3) A meaningful minimum-output protection. 4) Approval grants spending authority; the swap uses that authority to execute trading. 5) Pool state changes, transaction ordering, fees/route changes, or stale quote. 6) Better aggregate liquidity/output despite extra hops. 7) The quantities can change with pool state. 8) Onchain inclusion/execution result and logs.

24. Day 14 Homework and Portfolio Assignment

Homework A - Explain the AMM

Record a 2-minute explanation covering: DEX -> liquidity pool -> $x*y=k$ -> LP -> price impact -> slippage -> minimum output -> router. Use no notes if possible.

Homework B - Math worksheet

1. For reserves 500 TOKEN-A and 50,000 TOKEN-B, calculate k.
2. Ignoring fees, calculate output if 25 TOKEN-A enters.
3. Repeat using a 0.30% illustrative swap fee.
4. Calculate a 1% minimum-output floor from your quoted output.
5. Explain why the average execution price differs from the initial reserve ratio.

Homework C - Portfolio mini-project

Create a repository named defi-swap-analyzer. Include a small integer-based AMM quote simulator, a transaction-analysis template, sample decoded swap JSON from a test/fork environment, screenshots, test cases for slippage math, and a README that clearly says the simulator is educational and protocol-specific production integrations must use the exact official implementation.

Homework D - Client checklist

- Target chain + chain ID
- DEX/protocol + exact version
- Router/periphery addresses from official source
- Input/output tokens + decimals
- Quote source and route logic
- Approval/permit model
- Slippage policy
- MEV/privacy expectations
- Testing network/fork block
- Deployment/config ownership

25. Day 14 Cheat Sheet

DEX	= Onchain token exchange
AMM	= Algorithmic market maker using pooled liquidity
Liquidity pool	= Smart-contract reserves used for swaps
LP	= Liquidity provider / liquidity position owner
$x * y = k$	= Classic constant-product invariant
Spot/reserve ratio	= Current reserve-based price intuition
Price impact	= Price movement caused by your own trade
Slippage	= Quote-to-execution difference
amountOutMin	= Minimum output accepted for exact-input swap
amountInMax	= Maximum input accepted for exact-output swap
Router	= Periphery contract coordinating swap route
Factory/registry	= Pool creation/discovery layer
Pool/pair	= Holds liquidity and executes AMM accounting
Approval	= ERC-20 spending authorization
Path/route	= One or more pools used to reach output token
LP fee	= Fee economics assigned to liquidity under protocol rules
MEV	= Value extracted through transaction ordering/inclusion
Fork test	= Test against real deployed state at a controlled block

Five rules to remember

1. Never assume a DEX version from its brand name.
2. Never confuse price impact with slippage tolerance.
3. Never remove output protection just to make a reverting swap “work.”
4. Never trust token decimals, router addresses or pool state without verifying the target chain.
5. For production integration, use the exact protocol contracts/SDK/math and test on a pinned fork before mainnet deployment.

Day 15 Preview - Advanced DeFi Development

Next: staking and rewards, vesting, lending/borrowing architecture, flash loans, arbitrage mechanics, atomic execution, profitability constraints, and how to build a testable DeFi contract without confusing a demo with a production protocol.

26. Primary References and Instructor Notes

Use primary documentation when implementing against a specific protocol. AMM interfaces and fee/accounting details evolve, so these references should be rechecked at implementation time.

- Uniswap Developers - How Uniswap Works: <https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works>
- Uniswap Developers - Protocols Overview: <https://developers.uniswap.org/docs/protocols/overview>
- Uniswap Developers - Fees: <https://developers.uniswap.org/docs/get-started/concepts/fees>
- Uniswap Developers - v4 vs v3: <https://developers.uniswap.org/docs/protocols/v4/concepts/v4-vs-v3>
- Ethereum.org - DeFi: <https://ethereum.org/defi/>
- Ethereum.org - Maximal Extractable Value (MEV): <https://ethereum.org/developers/docs/mev/>
- Ethereum ERC-20 overview: <https://ethereum.org/developers/docs/standards/tokens/erc-20/>

Instructor whiteboard flow

```
wallet -> approve/permit -> quote -> slippage bound -> router
      -> pool(s) -> token transfers -> events/receipt -> UI/backend
```

For every client bug, ask:

CHAIN? VERSION? ADDRESS? TOKEN UNITS? ALLOWANCE? ROUTE?

QUOTE BLOCK? MIN OUTPUT? POOL STATE? RECEIPT? EVENTS?

Completion standard

Move to Day 15 when you can: calculate a basic $x*y=k$ swap, explain fees/price impact/slippage without mixing them up, trace wallet -> router -> pool -> output, explain LP positions at a beginner level, and debug a failed swap systematically from allowance through receipt/events.